

GUÍA PRÁCTICA #5

OPERACIONES: Conectando React con la API de Laravel (Fetch y Formularios).

PASO #1: Preparativos Críticos (El Puente entre Servidores).

Antes de programar en React, tener en cuenta que trabajaremos con **dos servidores corriendo al mismo tiempo** en sus computadoras:

1. **El Servidor Backend (Laravel):** En una terminal, deben estar en la carpeta de Laravel y ejecutar `php artisan serve` (usualmente corre en `http://127.0.0.1:8000`).
2. **El Servidor Frontend (Vite/React):** En otra terminal, deben estar en la carpeta de React y ejecutar `npm run dev` (usualmente corre en `http://localhost:5173`).

⚠ Nota sobre CORS: Asegúrate de que en Laravel el archivo `bootstrap/app.php` esté configurado para aceptar peticiones desde el puerto 5173 de Vite, de lo contrario el navegador bloqueará la conexión por seguridad (Realizar y/o revisar la Guía Práctica #4).

PASO 2: Generar los archivos de migración desde la terminal.

En la terminal, dentro de la carpeta del proyecto de Laravel (api-escuela), se deben ejecutar estos comandos en el siguiente orden para asegurar que los prefijos de marca de tiempo sigan la secuencia correcta:

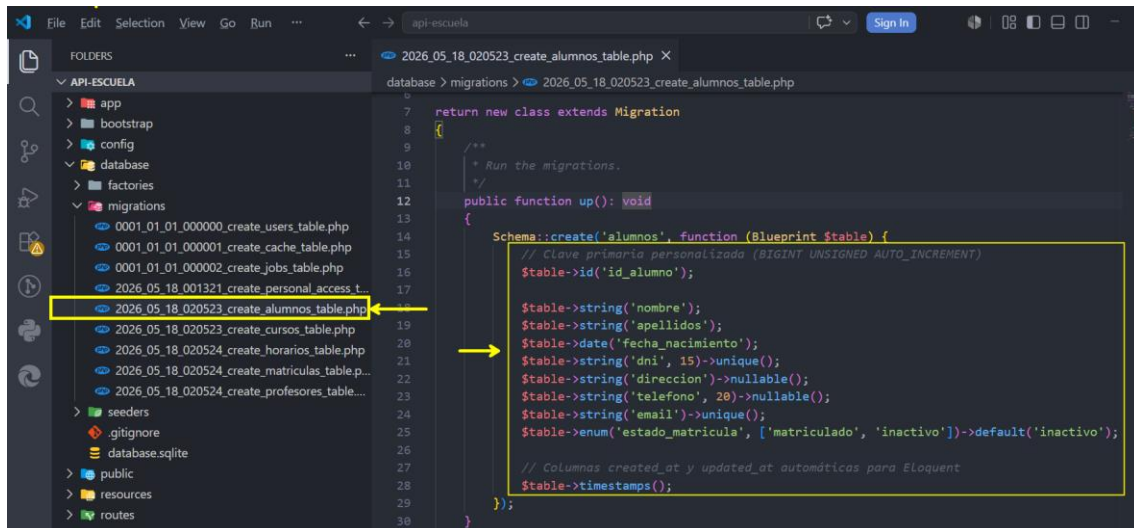
```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/api-escuela
$ php artisan make:migration create_alumnos_table
php artisan make:migration create_cursos_table
php artisan make:migration create_profesores_table
php artisan make:migration create_horarios_table
php artisan make:migration create_matriculas_table
```



PASO 3: Código de las Migraciones

Abre cada archivo creado en la carpeta **database/migrations/** y reemplaza el **método up()** con las siguientes estructuras:

1. Tabla Alumnos (create_alumnos_table.php)

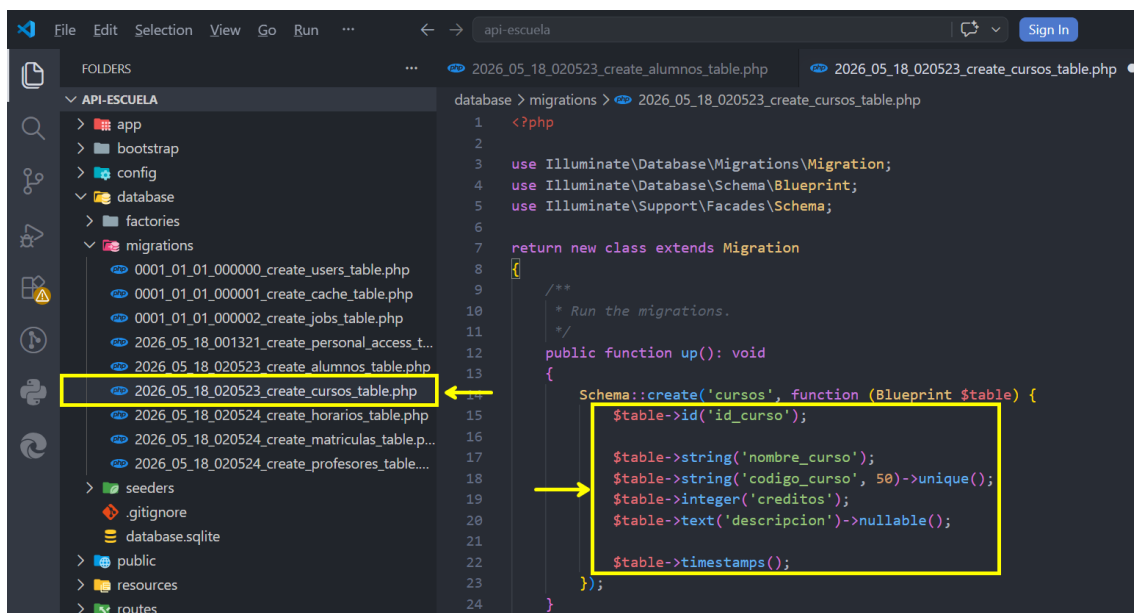


```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('alumnos', function (Blueprint $table) {
            // Clave primaria personalizada (BIGINT UNSIGNED AUTO_INCREMENT)
            $table->id('id_alumno');

            $table->string('nombre');
            $table->string('apellidos');
            $table->date('fecha_nacimiento');
            $table->string('dni', 15)->unique();
            $table->string('direccion')->nullable();
            $table->string('telefono', 20)->nullable();
            $table->string('email')->unique();
            $table->enum('estado_matricula', ['matriculado', 'inactivo'])->default('inactivo');

            // Columnas created_at y updated_at automáticas para Eloquent
            $table->timestamps();
        });
    }
};
```

2. Tabla Cursos (create_cursos_table.php)



```
<?php

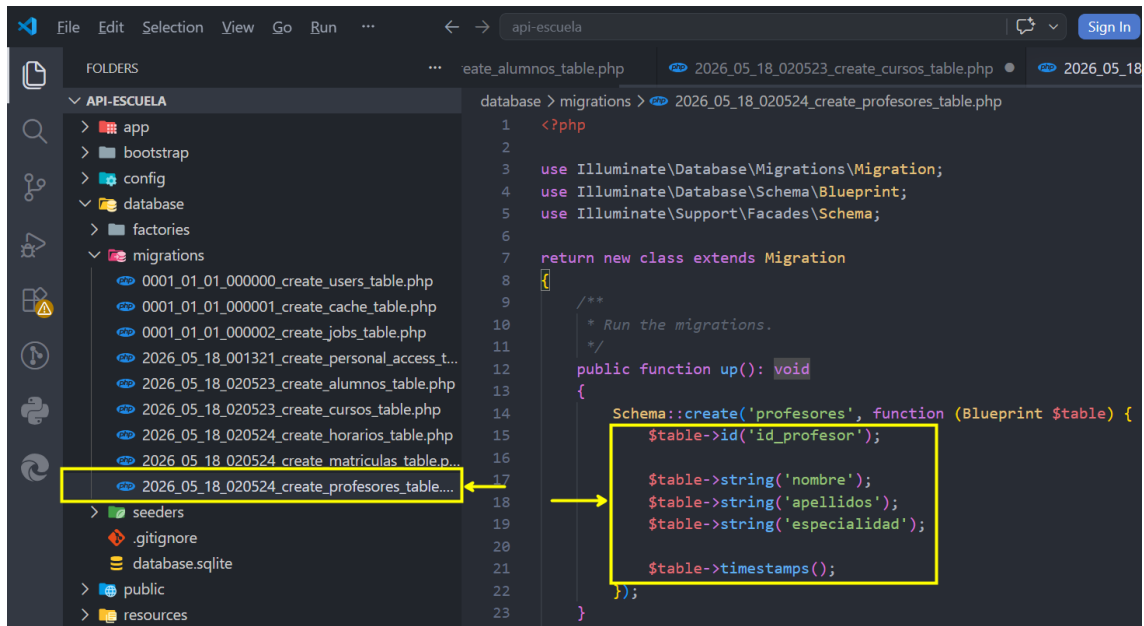
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('cursos', function (Blueprint $table) {
            $table->id('id_curso');

            $table->string('nombre_curso');
            $table->string('codigo_curso', 50)->unique();
            $table->integer('credits');
            $table->text('descripcion')->nullable();

            $table->timestamps();
        });
    }
};
```

3. Tabla Profesores (create_profesores_table.php)

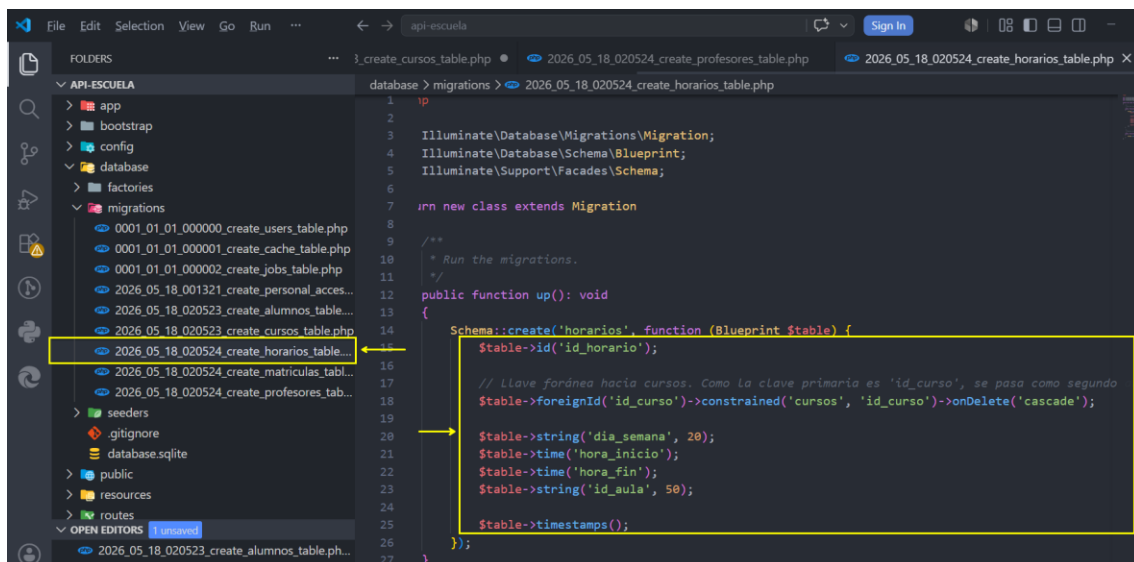


```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('profesores', function (Blueprint $table) {
15             $table->id('id_profesor');
16
17             $table->string('nombre');
18             $table->string('apellidos');
19             $table->string('especialidad');
20
21             $table->timestamps();
22         });
23     }

```

4. Tabla Horarios (create_horarios_table.php)

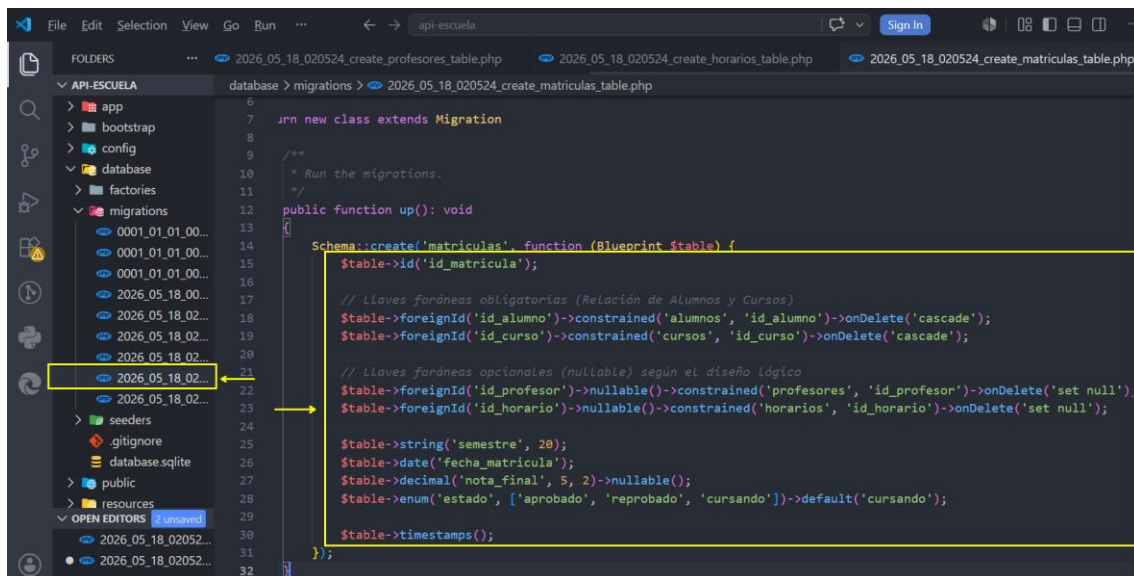


```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('horarios', function (Blueprint $table) {
15             $table->id('id_horario');
16
17             // Llave foránea hacia cursos. Como la clave primaria es 'id_curso', se pasa como segunda
18             $table->foreignId('id_curso')->constrained('cursos', 'id_curso')->onDelete('cascade');
19
20             $table->string('dia_semana', 20);
21             $table->time('hora_inicio');
22             $table->time('hora_fin');
23             $table->string('id_aula', 50);
24
25             $table->timestamps();
26         });
27     }

```

5. Tabla Matriculas (create_matriculas_table.php)



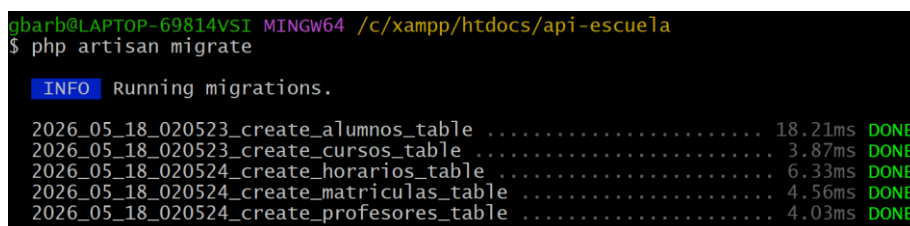
```

1  use Illuminate\Database\Migrations\Migration;
2  use Illuminate\Database\Schema\Blueprint;
3  use Illuminate\Support\Facades\Schema;
4
5  class CreateMatriculasTable extends Migration
6  {
7      /**
8       * Run the migrations.
9       */
10     public function up(): void
11     {
12         Schema::create('matriculas', function (Blueprint $table) {
13             $table->id('id_matricula');
14
15             // Llaves foráneas obligatorias (Relación de Alumnos y Cursos)
16             $table->foreignId('id_alumno')->constrained('alumnos', 'id_alumno')->onDelete('cascade');
17             $table->foreignId('id_curso')->constrained('cursos', 'id_curso')->onDelete('cascade');
18
19             // Llaves foráneas opcionales (nullable) según el diseño lógico
20             $table->foreignId('id_profesor')->nullable()->constrained('profesores', 'id_profesor')->onDelete('set null');
21             $table->foreignId('id_horario')->nullable()->constrained('horarios', 'id_horario')->onDelete('set null');
22
23             $table->string('semestre', 20);
24             $table->date('fecha_matricula');
25             $table->decimal('nota_final', 5, 2)->nullable();
26             $table->enum('estado', ['aprobado', 'reprobado', 'cursando'])->default('cursando');
27
28             $table->timestamps();
29         });
30     }
31 }

```

PASO 4: Ejecutar las migraciones

Una vez que todos los archivos estén guardados y la configuración de la base de datos esté lista en el archivo .env, ejecuta el siguiente comando en la terminal para crear físicamente toda la estructura relacional en MySQL:



```

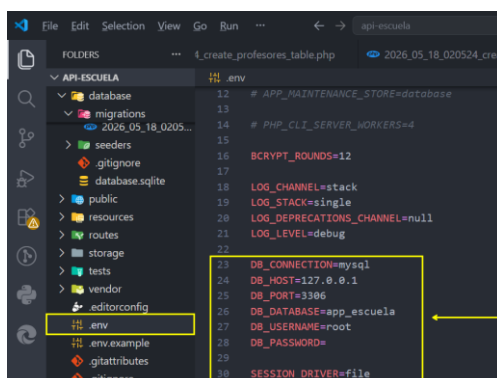
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/api-escuela
$ php artisan migrate

INFO Running migrations.

2026_05_18_020523_create_alumnos_table ..... 18.21ms DONE
2026_05_18_020523_create_cursos_table ..... 3.87ms DONE
2026_05_18_020524_create_horarios_table ..... 6.33ms DONE
2026_05_18_020524_create_matriculas_table ..... 4.56ms DONE
2026_05_18_020524_create_profesores_table ..... 4.03ms DONE

```

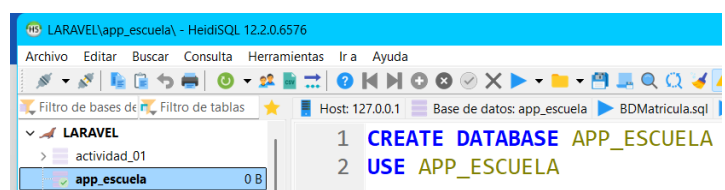
Sin embargo, configurar el archivo .env y crear la base de datos en HeidiSQL:



```

12 # APP_MAINTENANCE_STORE=database
13
14 # PHP_CLI_SERVER_WORKERS=4
15
16 BCRYPT_ROUNDS=12
17
18 LOG_CHANNEL=stack
19 LOG_STACK=single
20 LOG_DEPRECATED_CHANNEL=null
21 LOG_LEVEL=debug
22
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=app_escuela
27 DB_USERNAME=root
28 DB_PASSWORD=
29
30 SESSION_DRIVER=file

```



```

1 CREATE DATABASE APP_ESCUELA
2 USE APP_ESCUELA

```

Si sale el siguiente error:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/api-escuela
$ php artisan migrate

INFO Preparing database.

Creating migration table ..... 23.75ms DONE

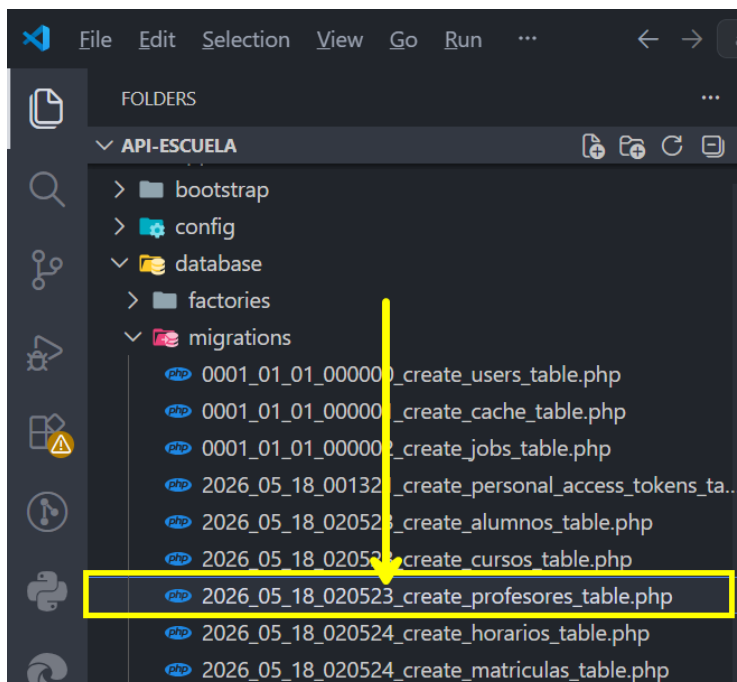
INFO Running migrations.

0001_01_01_000000_create_users_table ..... 67.05ms DONE
0001_01_01_000001_create_cache_table ..... 40.95ms DONE
0001_01_01_000002_create_jobs_table ..... 87.56ms DONE
2026_05_18_001321_create_personal_access_tokens_table ..... 39.46ms DONE
2026_05_18_020523_create_alumnos_table ..... 28.54ms DONE
2026_05_18_020523_create_cursos_table ..... 22.21ms DONE
2026_05_18_020524_create_horarios_table ..... 48.82ms DONE
2026_05_18_020524_create_matriculas_table ..... 114.72ms FAIL

Illuminate\Database\QueryException

SQLSTATE[HY000]: General error: 1005 Can't create table `app_escuela`.`matricu
las` (errno: 150 "Foreign key constraint is incorrectly formed") (Connection: my
sql, Host: 127.0.0.1, Port: 3306, Database: app_escuela, SQL: alter table `matri
culas` add constraint `matriculas_id_profesor_foreign` foreign key (`id_profesor`
) references `profesores` (`id_profesor`) on delete set null)
```

Esto se debe al orden de creación de las tablas. Primero debe existir Profesores antes que matricula y horarios. Cambiar la numeración.

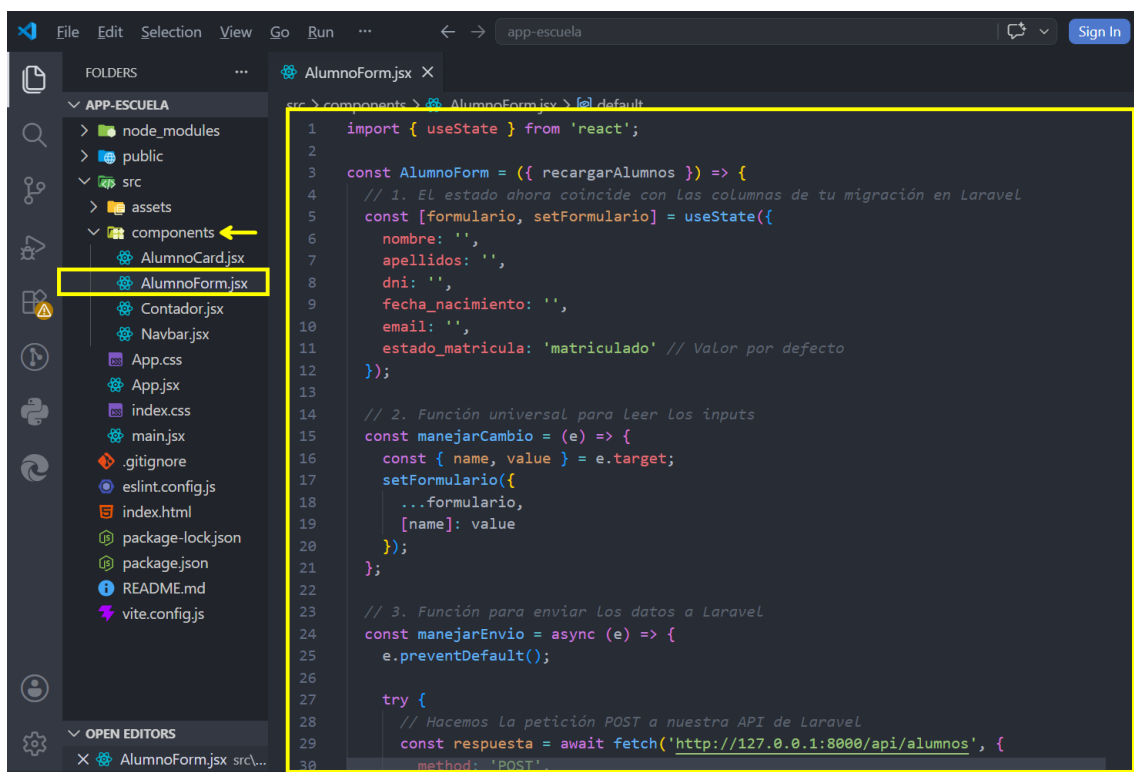


💡 Puntos clave para resaltar en la sesión práctica:

- **Convención de Constrained:** Cuando usamos nombres personalizados para las claves primarias (como `id_alumno` en lugar del clásico `id` de Laravel), estamos obligados a pasar el nombre de la tabla como primer argumento en `constrained()` y el nombre exacto de la columna PK como segundo argumento (ej. `constrained('alumnos', 'id_alumno')`).
- **Integridad Referencial (onDelete):** Mostrar la diferencia didáctica entre usar `onDelete('cascade')` (si se elimina un alumno, se borran automáticamente todas sus matrículas) y `onDelete('set null')` (si se elimina un profesor, la matrícula no se borra, simplemente el campo del profesor se queda vacío en NULL).

PASO 5: Adaptar el Formulario a la Base de Datos Real (Vamos a al proyecto de React – App-Escuela).

1. Abre `src/components/AlumnoForm.jsx`.
2. Actualiza el estado inicial y la lógica de envío (`manejarEnvio`) para incluir la petición fetch hacia Laravel:



```
1 import { useState } from 'react';
2
3 const AlumnoForm = ({ recargarAlumnos }) => {
4   // 1. El estado ahora coincide con las columnas de tu migración en Laravel
5   const [formulario, setFormulario] = useState({
6     nombre: '',
7     apellidos: '',
8     dni: '',
9     fecha_nacimiento: '',
10    email: '',
11    estado_matricula: 'matriculado' // Valor por defecto
12  });
13
14  // 2. Función universal para leer los inputs
15  const manejarCambio = (e) => {
16    const { name, value } = e.target;
17    setFormulario({
18      ...formulario,
19      [name]: value
20    });
21  };
22
23  // 3. Función para enviar los datos a Laravel
24  const manejarEnvio = async (e) => {
25    e.preventDefault();
26
27    try {
28      // Hacemos la petición POST a nuestra API de Laravel
29      const respuesta = await fetch('http://127.0.0.1:8000/api/alumnos', {
30        method: 'POST',
```



```
import { useState } from 'react';

const AlumnoForm = ({ recargarAlumnos }) => {
  // 1. El estado ahora coincide con las columnas de tu migración en
  // Laravel
  const [formulario, setFormulario] = useState({
    nombre: '',
    apellidos: '',
    dni: '',
    fecha_nacimiento: '',
    email: '',
    estado_matricula: 'matriculado' // Valor por defecto
  });

  // 2. Función universal para leer los inputs
  const manejarCambio = (e) => {
    const { name, value } = e.target;
    setFormulario({
      ...formulario,
      [name]: value
    });
  };

  // 3. Función para enviar los datos a Laravel
  const manejarEnvio = async (e) => {
    e.preventDefault();

    try {
      // Hacemos la petición POST a nuestra API de Laravel
      const respuesta = await fetch('http://127.0.0.1:8000/api/alumnos',
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json' // Fundamental para que Laravel
        devuelva errores en JSON
      },
      body: JSON.stringify(formulario)
    });

    const datos = await respuesta.json();

    if (respuesta.ok) {
      // Si Laravel responde con código 201 (Created)
      alert('Alumno guardado exitosamente en la base de datos');

      // Limpiamos el formulario
    }
  }
};
```



```

        setFormulario({
            nombre: '', apellidos: '', dni: '', fecha_nacimiento: '',
email: '', estado_matricula: 'matriculado'
        });

        // Llamamos a la función del padre (App.jsx) para que vuelva a
        pedir la lista actualizada a Laravel
        recargarAlumnos();
    } else {
        // Si hay errores de validación en Laravel (Código 422)
        console.log("Errores de validación:", datos.errors);
        alert('Error al guardar. Revisa la consola para más detalles.');
```

// Si hay errores de validación en Laravel (Código 422)

```

    }
} catch (error) {
    console.error('Error de conexión:', error);
    alert('No se pudo conectar con el servidor de Laravel.');
```

// Si hay errores de validación en Laravel (Código 422)

```

}
};

return (
    <div className="card shadow-sm mb-4">
        <div className="card-header bg-dark text-white">
            <h5 className="mb-0">Registrar Nuevo Alumno</h5>
        </div>
        <div className="card-body">
            <form onSubmit={manejarEnvio}>
                <div className="row g-3">
                    {/* Se construyen los inputs asegurando que el atributo
                    'name' coincida con el estado */}
                    <div className="col-md-6">
                        <label className="form-label">Nombres</label>
                        <input type="text" className="form-control" name="nombre"
value={formulario.nombre} onChange={manejarCambio} required />
                    </div>
                    <div className="col-md-6">
                        <label className="form-label">Apellidos</label>
                        <input type="text" className="form-control"
name="apellidos" value={formulario.apellidos} onChange={manejarCambio}
required />
                    </div>
                    <div className="col-md-4">
                        <label className="form-label">DNI</label>
                        <input type="text" className="form-control" name="dni"
value={formulario.dni} onChange={manejarCambio} required />
                    </div>
                    <div className="col-md-4">
                        <label className="form-label">Fecha de Nacimiento</label>

```




```

        <input type="date" className="form-control"
name="fecha_nacimiento" value={formulario.fecha_nacimiento}
onChange={manejarCambio} required />
      </div>
      <div className="col-md-4">
        <label className="form-label">Email</label>
        <input type="email" className="form-control" name="email"
value={formulario.email} onChange={manejarCambio} required />
      </div>
      <div className="col-md-4">
        <label className="form-label">Estado</label>
        <select className="form-select" name="estado_matricula"
value={formulario.estado_matricula} onChange={manejarCambio}>
          <option value="matriculado">Matriculado</option>
          <option value="inactivo">Inactivo</option>
        </select>
      </div>
      <div className="col-12 text-end mt-3">
        <button type="submit" className="btn btn-primary">
          <i className="fas fa-save me-2"></i>Guardar en MySQL
        </button>
      </div>
    </div>
  </form>
</div>
</div>
);
};

export default AlumnoForm;

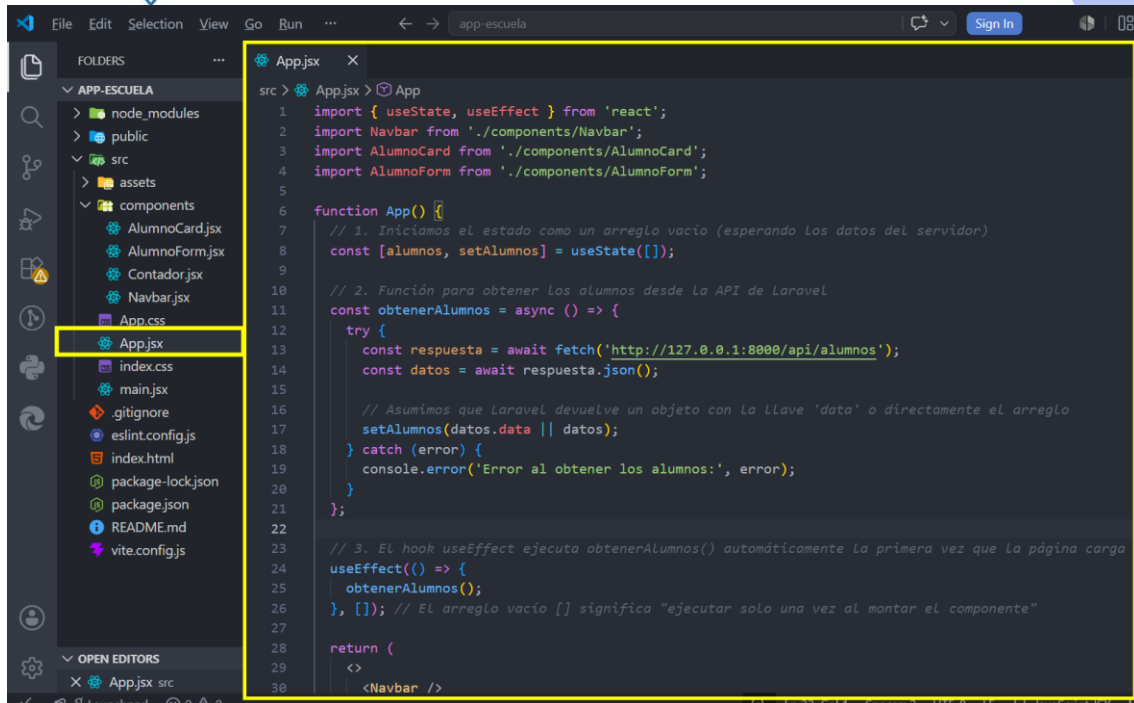
```

PASO 6: Traer los Datos Reales en App.jsx (Usando useEffect)

Ahora que el formulario guarda datos reales, nuestra lista principal ya no puede ser un arreglo estático escrito a mano. Debe conectarse a Laravel al momento de cargar la página para extraer los registros de MySQL.

1. Abre **src/App.jsx**.
2. Importa el hook **useEffect** e implementa la función para traer (GET) los datos:





The screenshot shows the VS Code editor with the 'app-escuela' project open. The file explorer on the left shows the project structure, with 'App.jsx' highlighted. The main editor displays the code for 'App.jsx'.

```

src > App.jsx > App
1  import { useState, useEffect } from 'react';
2  import Navbar from './components/Navbar';
3  import AlumnoCard from './components/AlumnoCard';
4  import AlumnoForm from './components/AlumnoForm';
5
6  function App() {
7    // 1. Iniciamos el estado como un arreglo vacío (esperando los datos del servidor)
8    const [alumnos, setAlumnos] = useState([]);
9
10   // 2. Función para obtener los alumnos desde la API de Laravel
11   const obtenerAlumnos = async () => {
12     try {
13       const respuesta = await fetch('http://127.0.0.1:8000/api/alumnos');
14       const datos = await respuesta.json();
15
16       // Asumimos que Laravel devuelve un objeto con la llave 'data' o directamente el arreglo
17       setAlumnos(datos.data || datos);
18     } catch (error) {
19       console.error('Error al obtener los alumnos:', error);
20     }
21   };
22
23   // 3. El hook useEffect ejecuta obtenerAlumnos() automáticamente la primera vez que la página carga
24   useEffect(() => {
25     obtenerAlumnos();
26   }, []); // El arreglo vacío [] significa "ejecutar solo una vez al montar el componente"
27
28   return (
29     <Navbar />
30

```

```

import { useState, useEffect } from 'react';
import Navbar from './components/Navbar';
import AlumnoCard from './components/AlumnoCard';
import AlumnoForm from './components/AlumnoForm';

function App() {
  // 1. Iniciamos el estado como un arreglo vacío (esperando los datos del servidor)
  const [alumnos, setAlumnos] = useState([]);

  // 2. Función para obtener los alumnos desde la API de Laravel
  const obtenerAlumnos = async () => {
    try {
      const respuesta = await fetch('http://127.0.0.1:8000/api/alumnos');
      const datos = await respuesta.json();

      // Asumimos que Laravel devuelve un objeto con la llave 'data' o directamente el arreglo
      setAlumnos(datos.data || datos);
    } catch (error) {
      console.error('Error al obtener los alumnos:', error);
    }
  };

  // 3. El hook useEffect ejecuta obtenerAlumnos() automáticamente la primera vez que la página carga
  useEffect(() => {
    obtenerAlumnos();
  }, []);

```



```

}, []); // El arreglo vacío [] significa "ejecutar solo una vez al
montar el componente"

return (
  <>
    <Navbar />
    <div className="container">
      <h2 className="mb-4">Directorio de Alumnos</h2>

      { /* Pasamos la función obtenerAlumnos al formulario para que
pueda recargar la tabla al guardar */ }
      <AlumnoForm recargarAlumnos={obtenerAlumnos} />

      <div className="row mt-4">
        {alumnos.map((alumno) => (
          <AlumnoCard
            key={alumno.id_alumno} // Usamos la llave primaria de la
base de datos
            nombre={` ${alumno.nombre} ${alumno.apellidos}`}
            carrera={alumno.email} // Temporal: Mostramos el email ya
que aún no cruzamos tablas
            estadoInicial={alumno.estado_matricula}
          />
        ))}
      </div>
    </div>
  </>
);
}

export default App;

```

💡 Momento Didáctico para el Laboratorio:

- **La función `async / await`:** Conectarse a una base de datos en otro servidor toma tiempo (milisegundos). React no puede congelar la pantalla esperando. Con `await`, le decimos al código: "Lanza la petición a Laravel, sigue haciendo otras cosas, y cuando Laravel responda, retoma esta línea y dibuja las tarjetas".



- **¿Por qué pasamos recargarAlumnos como Prop al formulario?** Porque cuando el formulario hace el POST y guarda el alumno, la base de datos se actualiza, pero la pantalla de React no se entera mágicamente. Al ejecutar recargarAlumnos() justo después del POST exitoso, forzamos a React a hacer un nuevo GET, trayendo la lista fresca desde MySQL e incluyendo al nuevo integrante al instante.

PASO 7: Ejecutar los servidores (Laravel y React).

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/api-escuela
$ php artisan serve

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server
```

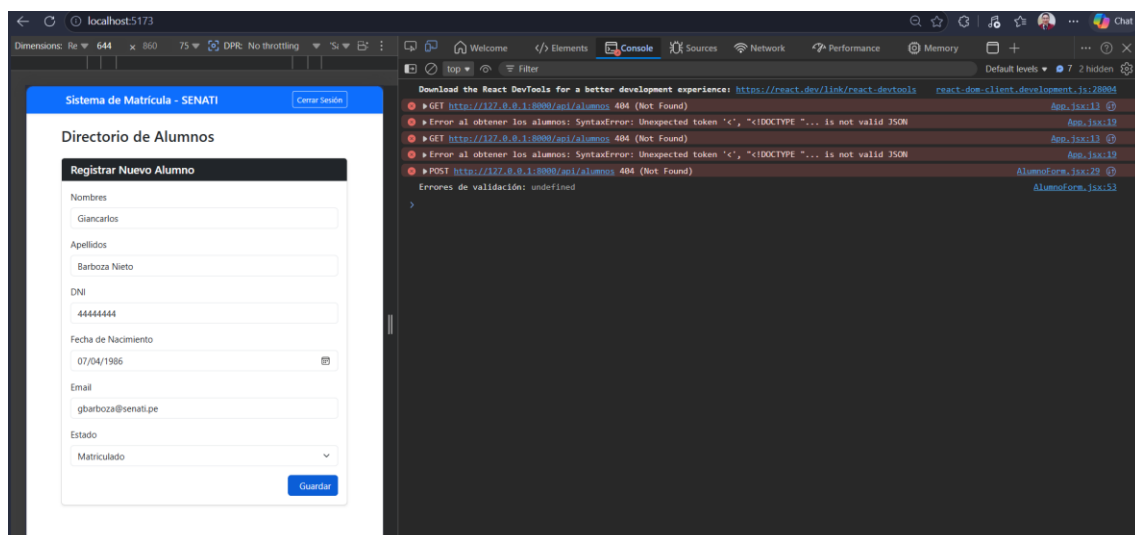
```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/app-escuela
$ npm run dev

> app-escuela@0.0.0 dev
> vite

VITE v8.0.13 ready in 247 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

PASO 8: Lo más probable es que salga un error.



Se presenta los siguientes errores:

1. **404 (Not Found)**: React fue a buscar la ruta `http://127.0.0.1:8000/api/alumnos`, pero Laravel le respondió: *"No tengo idea de qué es esa ruta, no existe"*.
2. **Unexpected token '<', "<!DOCTYPE "...**: Como Laravel no encontró la ruta, en lugar de devolver datos devolvió la clásica **página de error 404 en formato HTML**. Cuando React intentó leer ese HTML usando `respuesta.json()`, se tropezó con el primer carácter de la página (< de <!DOCTYPE html>) y colapsó porque esperaba llaves de JSON { o [.

El problema real: Construimos la base de datos y el frontend, pero nos faltó construir la "puerta de enlace" en Laravel: **El Modelo, el Controlador y la Ruta de la API**.

PASO 9: Configurar el Modelo Alumno

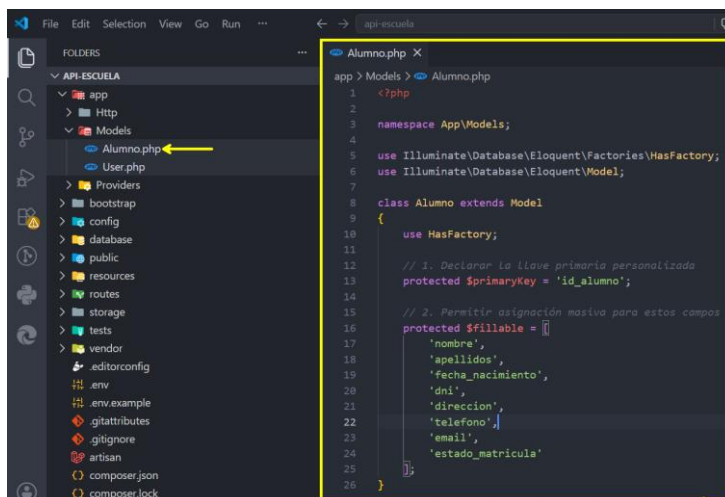
Laravel necesita saber qué campos permitimos guardar y cuál es nuestra llave primaria personalizada.

1. Abre el archivo **app/Models/Alumno.php** (si no existe, créalo en la terminal con `php artisan make:model Alumno`).

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/api-escuela
$ php artisan make:model Alumno

INFO Model [C:\xampp\htdocs\api-escuela\app\Models\Alumno.php] created successfully.
```

2. Configúralo exactamente así:



```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Alumno extends Model
{
    use HasFactory;

    // 1. Declarar la llave primaria personalizada
    protected $primaryKey = 'id_alumno';

    // 2. Permitir asignación masiva para estos campos
    protected $fillable = [
        'nombre',
        'apellidos',
        'fecha_nacimiento',
        'dni',
        'direccion',
        'telefono',
        'email',
        'estado_matricula'
    ];
}
```

Paso 9: Crear el Controlador de la API

Aquí es donde procesamos las peticiones GET (obtener la lista) y POST (guardar el alumno).

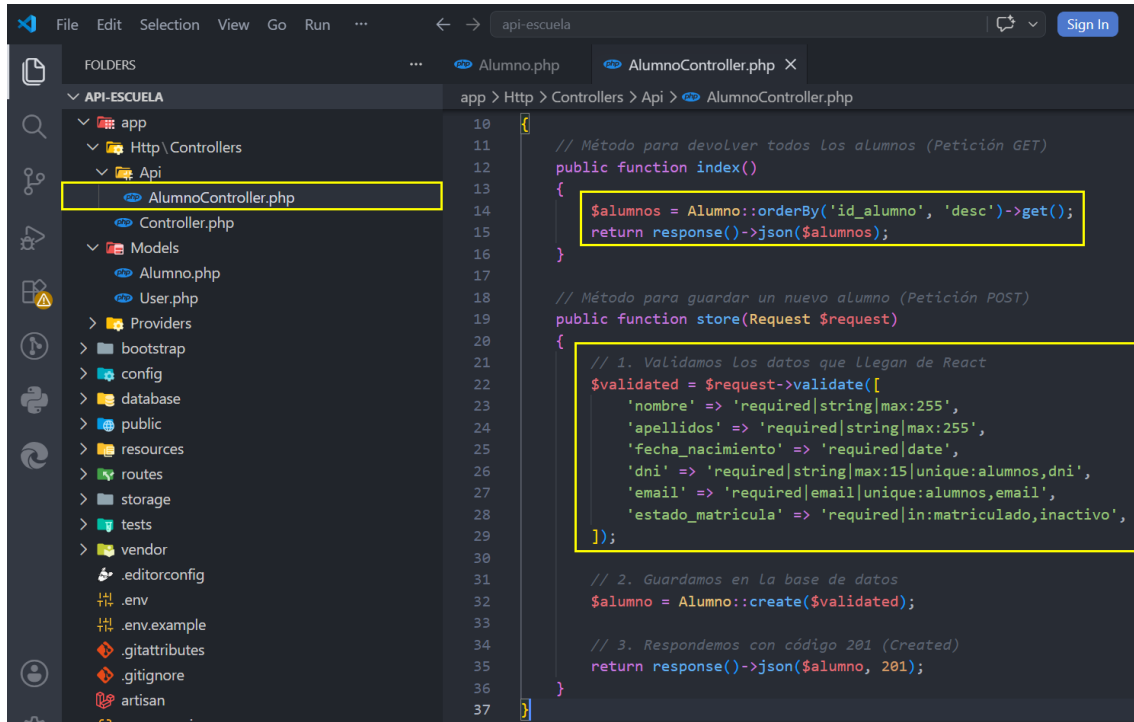
1. En la terminal de Laravel (api-escuela), ejecuta:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/api-escuela
$ php artisan make:controller Api/AlumnoController

INFO Controller [C:\xampp\htdocs\api-escuela\app\Http\Controllers\Api\AlumnoController.php] created successfully.
```



- Abre el archivo generado **app/Http/Controllers/Api/AlumnoController.php** y agrega los métodos **index** y **store**:



```

10  {
11  // Método para devolver todos Los alumnos (Petición GET)
12  public function index()
13  {
14      $alumnos = Alumno::orderBy('id_alumno', 'desc')->get();
15      return response()->json($alumnos);
16  }
17
18  // Método para guardar un nuevo alumno (Petición POST)
19  public function store(Request $request)
20  {
21      // 1. Validamos Los datos que Llegan de React
22      $validated = $request->validate([
23          'nombre' => 'required|string|max:255',
24          'apellidos' => 'required|string|max:255',
25          'fecha_nacimiento' => 'required|date',
26          'dni' => 'required|string|max:15|unique:alumnos,dni',
27          'email' => 'required|email|unique:alumnos,email',
28          'estado_matricula' => 'required|in:matriculado,inactivo',
29      ]);
30
31      // 2. Guardamos en La base de datos
32      $alumno = Alumno::create($validated);
33
34      // 3. Respondemos con código 201 (Created)
35      return response()->json($alumno, 201);
36  }
37  }
  
```

```

<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Alumno;
use Illuminate\Http\Request;

class AlumnoController extends Controller
{
    // Método para devolver todos Los alumnos (Petición GET)
    public function index()
    {
        $alumnos = Alumno::orderBy('id_alumno', 'desc')->get();
        return response()->json($alumnos);
    }

    // Método para guardar un nuevo alumno (Petición POST)
    public function store(Request $request)
    {
        // 1. Validamos Los datos que Llegan de React
        $validated = $request->validate([
            'nombre' => 'required|string|max:255',
  
```



```
'apellidos' => 'required|string|max:255',
'fecha_nacimiento' => 'required|date',
'dni' => 'required|string|max:15|unique:alumnos,dni',
'email' => 'required|email|unique:alumnos,email',
'estado_matricula' => 'required|in:matriculado,inactivo',
]);

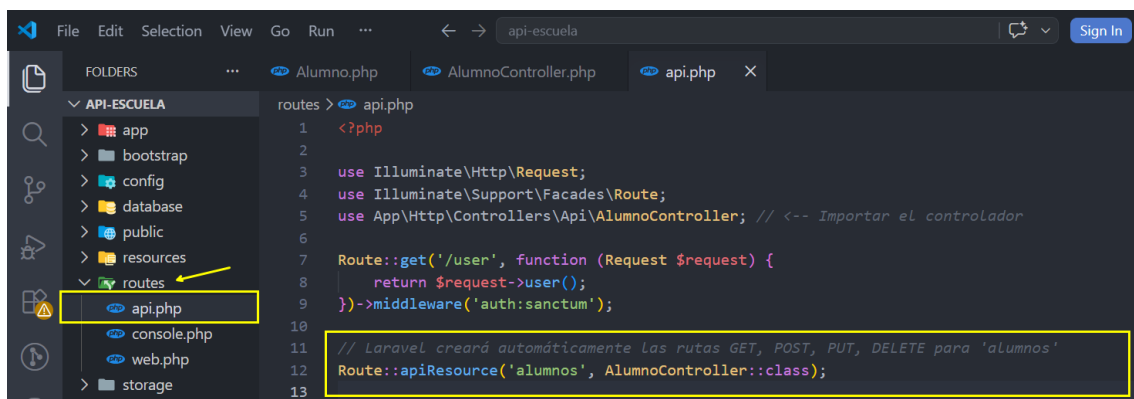
// 2. Guardamos en la base de datos
$alumno = Alumno::create($validated);

// 3. Respondemos con código 201 (Created)
return response()->json($alumno, 201);
}
}
```

Paso 10: Exponer la Ruta en api.php

Este es el paso que elimina el error 404. Le diremos a Laravel que encienda esa ruta.

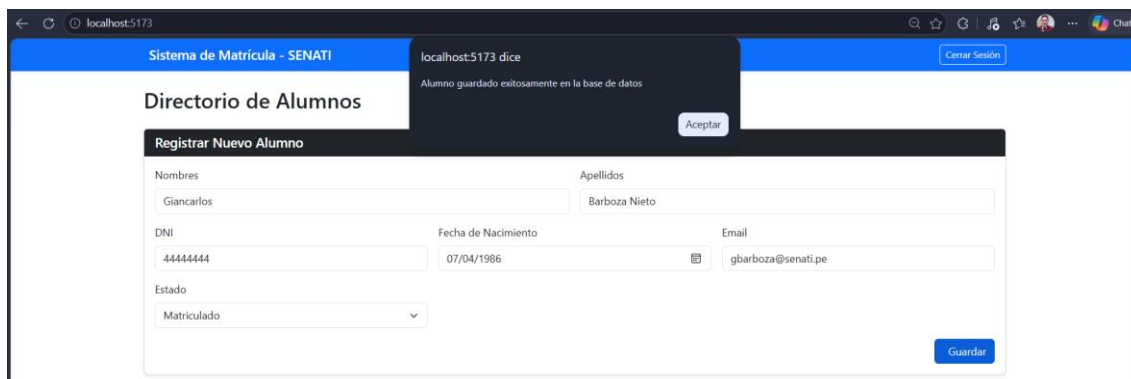
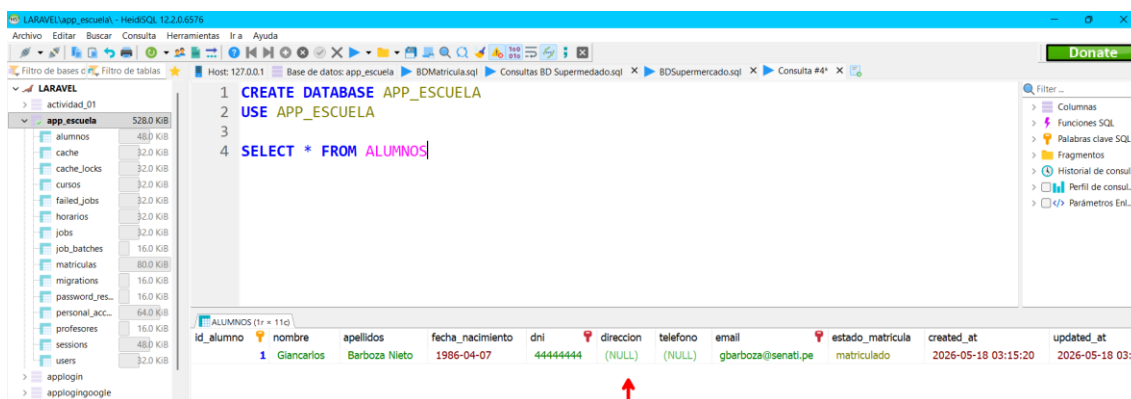
1. Abre el archivo **routes/api.php**.
2. Agrega la ruta apuntando a tu nuevo controlador:



3. Ejecutar los servidores (Laravel y React).



¡Felicitaciones! Los datos ingresados en el formulario han sido guardados exitosamente en la base de datos MySQL.

id_alumno	nombre	apellidos	fecha_nacimiento	dni	direccion	telefono	email	estado_matricula	created_at	updated_at
1	Giancarlos	Barboza Nieto	1986-04-07	44444444	(NULL)	(NULL)	gbarboza@senati.pe	matriculado	2026-05-18 03:15:20	2026-05-18 03:15:20

